



2A-SN – Ingénierie Dirigée par les Modèles

Projet 2025-2026

Spécification et Validation de Circuits Électroniques

Rapport de projet

Groupe B5

Bellahbib mhamed el ghali

Brahim Jedou Cheikh

El-Boubekri Oussama

Contents

1	Introduction	4
2	Architecture de la solution	4
2.1	Vue d'ensemble	4
3	Métamodèle Catalog	5
3.1	Objectif	5
3.2	Diagramme Ecore	5
3.3	Classes principales	5
4	Métamodèle Netlist	6
4.1	Objectif	6
4.2	Diagramme Ecore	6
4.3	Classes principales	6
5	Métamodèle Layout	7
5.1	Objectif	7
5.2	Diagramme Ecore	7
5.3	Classes principales	7
6	Contraintes de validation	8
6.1	Objectif des contraintes	8
6.2	Validateur du métamodèle Catalog	8
6.2.1	Classe CatalogValidator	8
6.3	Validateur du métamodèle Netlist	8
6.3.1	Classe NetlistValidator	9
6.4	Validateur du métamodèle Layout	9
6.4.1	Classe LayoutValidator	9
6.5	Bénéfices de la validation	10
7	Éditeurs graphiques (Sirius)	11
7.1	Éditeur de Netlist	11
8	Acceleo	12
9	Génération de la BOM (Bill of Materials)	13

10 Description du rendu	13
10.1 Organisation générale	13
10.2 Projets principaux	14
10.2.1 fr.n7.idm.catalog	14
10.2.2 fr.n7.idm.netlist	14
10.2.3 fr.n7.idm.layout	14
10.2.4 fr.n7.idm.bom.acceleo	15
10.2.5 fr.n7.idm.design	15
10.2.6 fr.n7.idm.catalogatl.atl	15
10.2.7 fr.n7.idm.netlist.acceleo	15
10.2.8 fr.n7.idm.catalog.acceleo	15
10.2.9 fr.n7.idm.catalog.xtext	16
10.3 Projets de tests	16
10.3.1 fr.n7.idm.tests	16
10.3.2 fr.n7.idm.test2	16
11 Syntaxe textuelle (Xtext)	17
11.1 Exemple de définition de catalogue	17
12 Couverture fonctionnelle	18
13 Conclusion	19
13.1 Réalisations	19
13.2 Apports de l'approche MDE	19
13.3 Limitations et perspectives	20
13.4 Bilan	20

List of Figures

1	Diagramme Ecore du métamodèle Catalog	5
2	Diagramme Ecore du métamodèle Netlist	6
3	Diagramme Ecore du métamodèle Layout	7
4	Éditeur Sirius pour la Netlist	11
5	Exemple de circuit avec microprocesseur, résistance et ventilateur netlist.dot	12
6	Exemple de Catalog.dot	12
7	BOM générée automatiquement à partir de la netlist	13

1 Introduction

Ce projet s'inscrit dans le cadre de l'unité d'enseignement *Ingénierie Dirigée par les Modèles*. Il vise à concevoir une suite d'outils permettant la modélisation et la validation de circuits électroniques complexes.

L'objectif principal est de fournir aux ingénieurs électroniciens des outils permettant de :

- Définir des catalogues de composants avec leurs contraintes
- Créer des circuits (netlist) en connectant logiquement les composants
- Concevoir le placement physique (layout) sur les cartes électroniques
- Valider automatiquement la cohérence et le respect des contraintes

Notre solution repose sur trois métamodèles complémentaires : **Catalog**, **Netlist** et **Layout**. Ces métamodèles sont interconnectés par des références croisées EMF et s'accompagnent d'éditeurs graphiques (Sirius) et textuels (Xtext).

2 Architecture de la solution

2.1 Vue d'ensemble

Notre architecture sépare trois aspects fondamentaux :

- **Catalog** : bibliothèque de composants réutilisables
- **Netlist** : schéma électrique (connexions logiques)
- **Layout** : disposition physique sur le PCB

Cette séparation offre réutilisabilité, maintenabilité et facilite le travail collaboratif.

3 Métamodèle Catalog

3.1 Objectif

Le métamodèle Catalog définit les types de composants électroniques avec leurs caractéristiques (métadonnées, empreintes, ports) et leurs contraintes de placement.

3.2 Diagramme Ecore

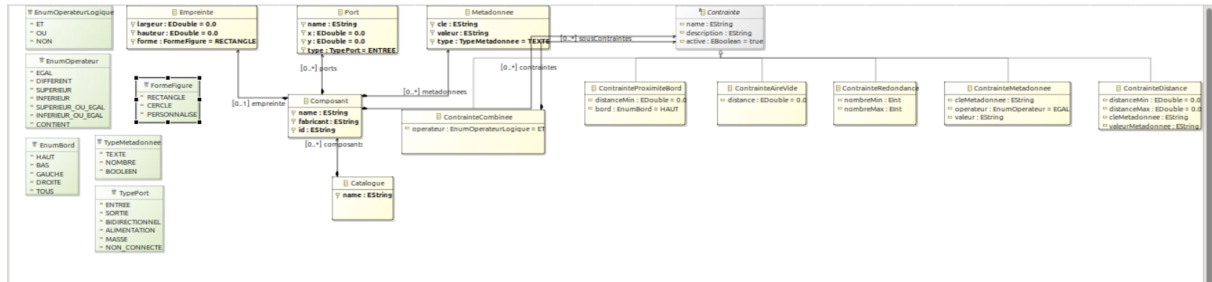


Figure 1: Diagramme Ecore du métamodèle Catalog

3.3 Classes principales

- **Catalogue** : conteneur de composants
- **Composant** : type de composant (nom, fabricant, ID, métadonnées)
- **Empreinte** : forme physique (largeur, hauteur, forme)
- **Port** : points de connexion (x, y, type)
- **Contrainte** : hiérarchie de contraintes (AireVide, Distance, ProximiteBord, Redondance, Meta-donnee, Combinee)

Le système de contraintes permet d'exprimer des règles complexes comme : "(ContrainteAireVide OU ContrainteDistance) ET ContrainteProximiteBord".

4 Métamodèle Netlist

4.1 Objectif

Le métamodèle Netlist décrit les connexions logiques entre composants, indépendamment de leur placement physique.

4.2 Diagramme Ecore

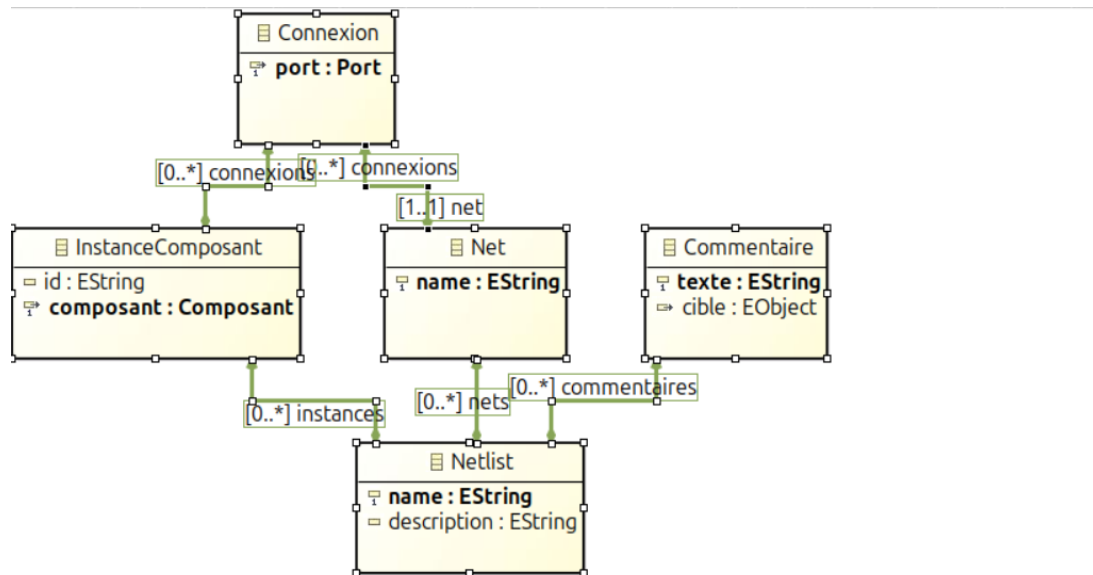


Figure 2: Diagramme Ecore du métamodèle Netlist

4.3 Classes principales

- **Netlist** : conteneur du circuit
- **InstanceComposant** : instance concrète d'un composant (ex: C1, C2, C3)
- **Net** : réseau électrique reliant plusieurs ports
- **Connexion** : lien entre un port et un net
- **Commentaire** : annotations sur le circuit

La Netlist importe le métamodèle Catalog pour référencer les composants et ports.

5 Métamodèle Layout

5.1 Objectif

Le métamodèle Layout décrit le placement physique des composants sur les cartes électroniques (PCB).

5.2 Diagramme Ecore

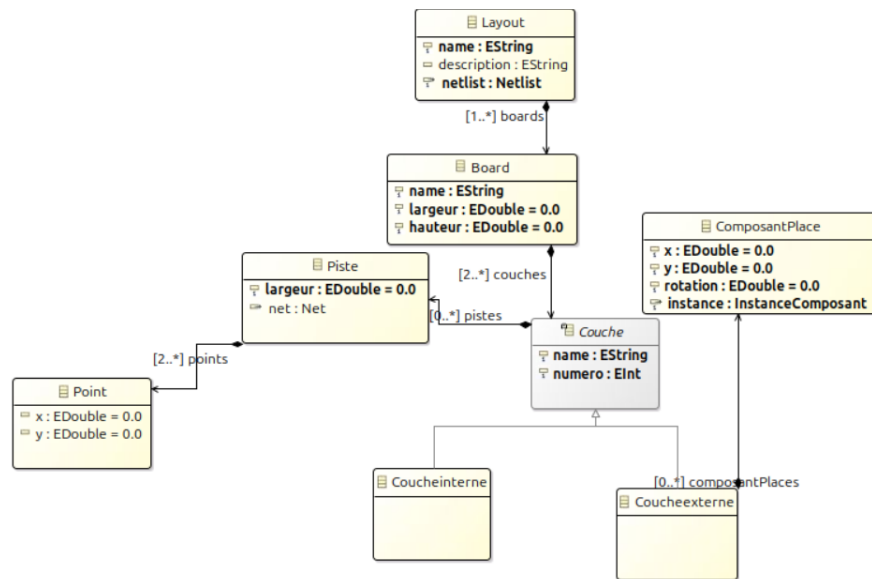


Figure 3: Diagramme Ecore du métamodèle Layout

5.3 Classes principales

- **Layout** : conteneur avec boards et référence à la netlist
- **Board** : carte physique (dimensions en mm)
- **Couche** : abstraite avec deux spécialisations
 - *Coucheexterne* : peut contenir des composants (max 2 par board)
 - *Coucheinterne* : seulement des pistes
- **ComposantPlace** : composant positionné (x, y, rotation)
- **Piste** : trace de cuivre avec points de passage
- **Point** : coordonnées (x, y)

Le Layout référence la Netlist pour garantir la cohérence entre schéma logique et implémentation physique.

6 Contraintes de validation

6.1 Objectif des contraintes

Les contraintes de validation permettent de garantir la cohérence et la validité des modèles au-delà des règles structurelles définies par les métamodèles Ecore. Elles vérifient les règles métier spécifiques au domaine de la conception électronique et détectent les erreurs de modélisation avant la génération de code ou la fabrication physique.

Nous avons implémenté des **validateurs Java** pour les trois métamodèles, en créant des classes héritant de `EValidator`. Ces validateurs sont automatiquement invoqués par l'environnement EMF lors de la validation des modèles.

6.2 Validateur du métamodèle Catalog

Le métamodèle Catalog nécessite des règles strictes pour garantir la cohérence des composants réutilisables.

6.2.1 Classe `CatalogValidator`

Le tableau suivant présente les principales contraintes implémentées :

Contrainte	Description et justification
Unicité des noms de composants	Deux composants dans un même catalogue ne peuvent avoir le même nom. Évite les ambiguïtés lors de l'instanciation.
Unicité des identifiants	Chaque composant doit avoir un identifiant unique (ex: "RES001"). Sert de référence absolue pour la traçabilité.
Nom non vide	Le nom d'un composant ne peut pas être vide ou null. Un composant doit être identifiable.
Empreinte obligatoire	Chaque composant doit avoir une empreinte définie. Sans empreinte, impossible de placer le composant sur un PCB.
Dimensions positives	La largeur et la hauteur d'une empreinte doivent être strictement positives. Dimensions nulles ou négatives sans sens physique.
Nombre minimal de ports	Un composant doit avoir au minimum deux ports. Nécessaire pour être utilisable dans un circuit.
Unicité des noms de ports	Les ports d'un même composant doivent avoir des noms uniques (ex: VCC, GND, OUT1). Permet l'identification sans ambiguïté.
Coordonnées des ports valides	Les coordonnées (x, y) de chaque port doivent être à l'intérieur de l'empreinte. Un port ne peut être hors du composant.
Métadonnées valides	Les clés des métadonnées ne peuvent pas être vides. Une métadonnée sans clé est inexploitable.
Contraintes géométriques positives	Les valeurs de distance et de rayon doivent être strictement positives. Distance ou rayon négatif sans sens physique.

6.3 Validateur du métamodèle Netlist

Le métamodèle Netlist doit garantir la cohérence des connexions électriques entre composants.

6.3.1 Classe NetlistValidator

Le tableau suivant présente les principales contraintes implémentées :

Contrainte	Description et justification
Netlist non vide	Une netlist doit contenir au moins une instance de composant. Une netlist vide n'a pas de sens fonctionnel.
Unicité des identifiants d'instances	Chaque instance doit avoir un identifiant unique (ex: R1, R2, C1). Permet le référencement sans ambiguïté.
Instance référence un composant	Chaque instance doit référencer un composant valide du catalogue. On ne peut instancier que des composants existants.
Unicité des noms de nets	Chaque réseau électrique (net) doit avoir un nom unique (ex: VCC, GND). Identification sans ambiguïté.
Net connecte au moins deux ports	Un net doit relier au minimum deux ports. Un net avec moins de deux connexions est inutile.
Connexions valides	Chaque connexion doit lier un port existant à un net existant. Évite les connexions orphelines.
Port appartient à une instance	Un port connecté doit appartenir à un composant présent dans le circuit. On ne peut connecter des ports absents.
Pas de port connecté plusieurs fois	Un port physique ne peut être relié qu'à un seul net. Évite les court-circuits logiques.
Détection des instances isolées	Avertissement si une instance n'est connectée à aucun net. Probablement une erreur de conception.
Cohérence des types de ports	Un net ne doit pas connecter uniquement des ports SORTIE. Créerait un court-circuit.

6.4 Valideur du métamodèle Layout

Le métamodèle Layout vérifie les règles de placement physique sur les cartes PCB.

6.4.1 Classe LayoutValidator

Le tableau suivant présente les principales contraintes implémentées :

Contrainte	Description et justification
Au moins un board	Un layout doit contenir au moins une carte (board). Sans board, impossible de placer des composants.
Dimensions positives du board	La largeur et la hauteur d'un board doivent être strictement positives. Dimensions physiques valides requises.
Maximum 2 couches externes	Un board peut avoir au maximum 2 couches externes. Un PCB standard a une face TOP et BOTTOM.
Composant dans les limites	Un composant placé ne peut pas dépasser des limites du board. Tout dépassement serait coupé lors de la fabrication.
Rotation valide	L'angle de rotation doit être compris entre 0° et 360°. Convention standard pour l'orientation.
Pas de chevauchement	Deux composants sur la même couche ne peuvent pas se chevaucher. Deux composants ne peuvent occuper le même espace.
Points de piste dans le board	Tous les points d'une piste doivent être à l'intérieur du board. Une piste ne peut sortir de la carte.
Piste relie au moins deux points	Une piste doit contenir au minimum deux points. Une piste avec un seul point ne connecte rien.
Composants sur couches externes	Les couches internes ne peuvent pas contenir de composants. Composants montés uniquement sur faces externes.
Cohérence avec la netlist	Toutes les instances de la netlist doivent être placées dans le layout. Composant logique doit avoir un placement physique.
Validation des contraintes catalogue	Les contraintes (aire vide, distance, proximité bords) doivent être respectées. Garantit le respect des règles de conception.

6.5 Bénéfices de la validation

Les validateurs Java que nous avons implémentés permettent de :

- **Détecter précocement les erreurs** : Les violations sont signalées dès la modélisation, avant génération ou fabrication
- **Garantir la cohérence inter-modèles** : Les références entre Catalog, Netlist et Layout sont validées
- **Respecter les règles métier** : Les contraintes électroniques (distances, aires de sécurité) sont vérifiées automatiquement
- **Faciliter la maintenance** : Les contraintes sont centralisées dans des classes dédiées
- **Améliorer la qualité** : Réduction drastique des erreurs de conception grâce à la validation automatique
- **Feedback immédiat** : Intégration native dans l'éditeur Eclipse avec marqueurs d'erreurs

Cette approche de validation par validateurs Java offre une grande flexibilité pour implémenter des règles complexes tout en bénéficiant de l'intégration native avec l'écosystème Eclipse/EMF.

7 Éditeurs graphiques (Sirius)

Nous avons développé des éditeurs graphiques avec Sirius pour faciliter la création et la visualisation des circuits.

7.1 Éditeur de Netlist

L'éditeur Sirius pour la Netlist permet de créer visuellement des circuits électroniques en manipulant directement les éléments du métamodèle. L'interface graphique offre une **palette d'outils** complète permettant de :

- **Créer des instances de composants** : L'utilisateur sélectionne un composant du catalogue et le place sur le diagramme. Chaque instance (ex: R1, C1, IC1) est automatiquement associée à son composant référence avec tous ses ports.
- **Créer des nets (nœuds électriques)** : Les nets représentent les connexions logiques entre composants. Ils sont visualisés sous forme de nœuds intermédiaires permettant de relier plusieurs ports ensemble.
- **Établir des connexions** : Des liens visuels (traits) relient les ports des instances aux nets, matérialisant le schéma électrique. Ces connexions sont automatiquement traduites en objets Connexion conformes au métamodèle.
- **Ajouter des commentaires** : Des annotations textuelles peuvent être attachées aux instances ou au circuit global pour documenter la conception.

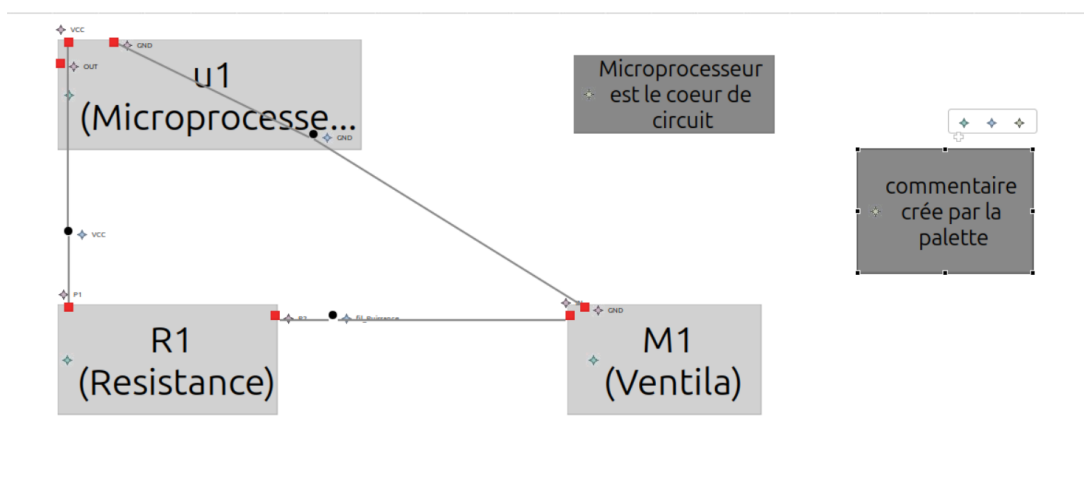


Figure 4: Éditeur Sirius pour la Netlist

L'exemple de la Figure 4 illustre un circuit simple composé d'un microprocesseur (u1), d'une résistance (R1) et d'un ventilateur (M1), interconnectés via des nets. Les connexions sont représentées par des traits reliant les ports (marqués par des petits carrés rouges) aux nœuds de connexion. Un commentaire explicatif est visible sur la droite du diagramme.

Cette interface graphique facilite considérablement la conception de circuits par rapport à une saisie textuelle ou arborescente, tout en maintenant la rigueur du métamodèle sous-jacent.

L'éditeur Sirius permet de créer visuellement les circuits en plaçant les composants et en les reliant. Les connexions sont automatiquement traduites en modèle conforme au métamodèle Netlist.

8 Acceleo

Dans les fichiers Acceleo, nous avons implémenté des règles de transformation (M2T) pour générer automatiquement du code Graphviz à partir de nos modèles. Concrètement, nous avons programmé la production de deux types de fichiers .dot distincts : le premier est destiné à la visualisation du catalogue, affichant de manière exhaustive tous les composants disponibles avec leurs détails structuraux (pins, empreintes, métadonnées et contraintes) ; le second fichier est dédié à la visualisation de la netlist, permettant de représenter graphiquement la topologie du circuit final et les connexions entre les différentes instances de composants.

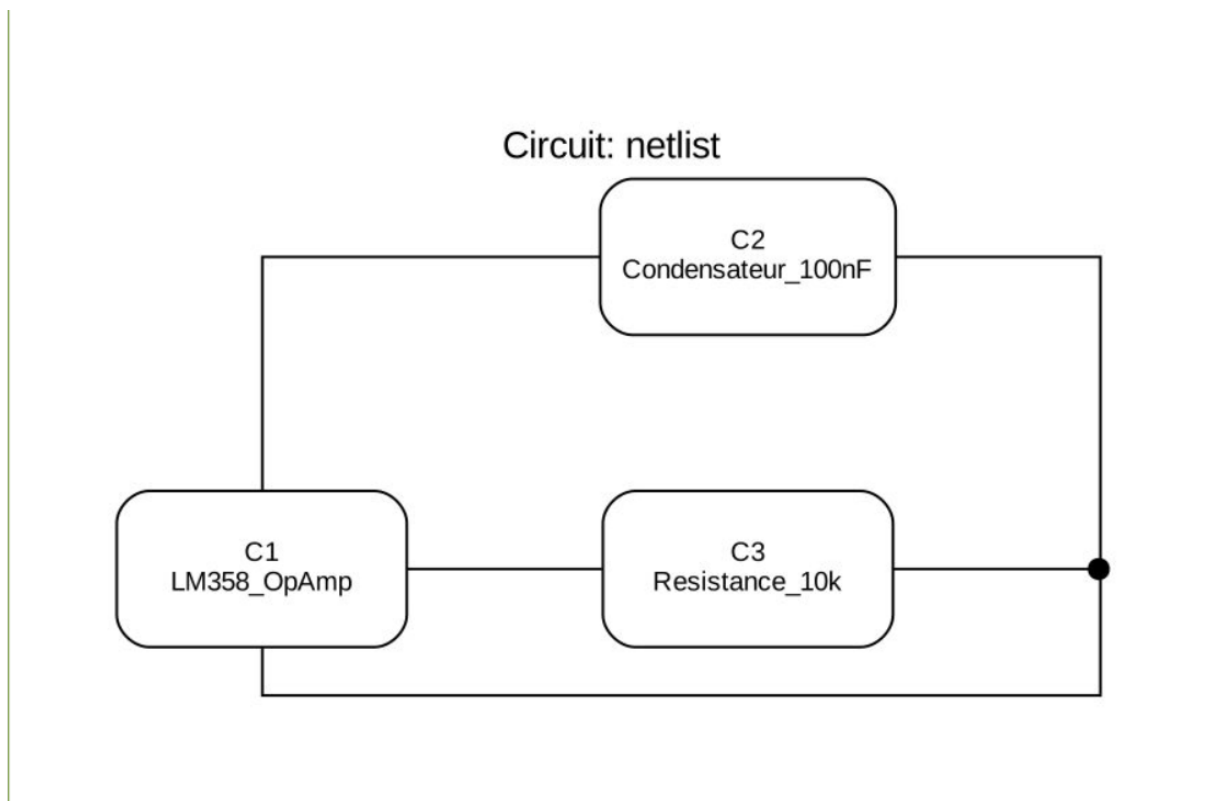


Figure 5: Exemple de circuit avec microprocesseur, résistance et ventilateur netlist.dot

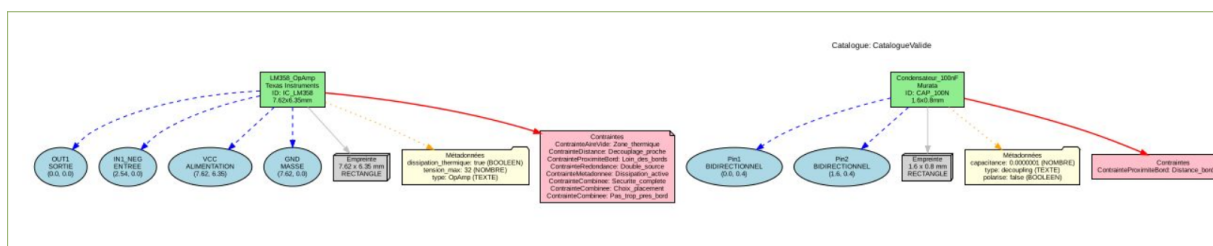


Figure 6: Exemple de Catalog.dot

9 Génération de la BOM (Bill of Materials)

À partir d'une netlist, notre outil génère automatiquement une nomenclature (BOM) listant tous les composants nécessaires.

NOMENCLATURE (BOM) Circuit: netlist Date: 2026-01-04				
Référence	Composant	Fabricant	ID Comp	Empreinte
C1	LM358_OpAmp	Texas Instruments	IC_LM358	7.62x6.35mm RECTANGLE
C2	Condensateur_100nF	Murata	CAP_100N	1.6x0.8mm RECTANGLE
C3	Resistance_10k	Vishay	RES_10K	2.0x1.25mm RECTANGLE
TOTAL			3 composants	
MÉTADONNÉES DES COMPOSANTS				
LM358_OpAmp	dissipation_thermique: true (BOOLEEN) tension_max: 32 (NOMBRE) type: OpAmp (TEXTE)			
Resistance_10k	resistance: 10000 (NOMBRE) tolerance: 5 (NOMBRE) puissance_max: 0.125 (NOMBRE)			
Condensateur_100nF	capacitance: 0.0000001 (NOMBRE) type: decoupling (TEXTE) polarise: false (BOOLEEN)			

Figure 7: BOM générée automatiquement à partir de la netlist

La BOM inclut :

- Référence de chaque composant dans le circuit
- Type de composant avec son nom
- Fabricant
- Identifiant unique du composant
- Dimensions de l'empreinte
- Métadonnées importantes (valeur, tolérance, type, etc.)

Cette génération automatique facilite la commande des composants et la vérification des quantités.

10 Description du rendu

10.1 Organisation générale

Le rendu est organisé en plusieurs projets Eclipse correspondant aux différents aspects du projet :

- Projets de métamodèles (Catalog, Netlist, Layout)
- Projets de transformations (Acceleo pour génération BOM, Netlist.dot, Catalog.dot)

- Projets d'éditeurs graphiques (Sirius) eclipse de déploiement
- Projets ATL (eclipse de déploiement)
- Projets XText
- Projets de tests et exemples

10.2 Projets principaux

10.2.1 fr.n7.idm.catalog

Description : Métamodèle Catalog définissant les composants électroniques.

Fichiers importants :

- `model/catalog.ecore` : métamodèle Ecore
- `model/catalog.genmodel` : modèle de génération
- `src/` : code Java généré par EMF
- Exemple valide et non valide du catalog

10.2.2 fr.n7.idm.netlist

Description : Métamodèle Netlist pour les circuits logiques.

Fichiers importants :

- `model/netlist.ecore` : métamodèle (importe `catalog.ecore`)
- `model/netlist.genmodel` : génération (référence `catalog.genmodel`)
- `src/` : code Java généré par EMF
- Exemple valide et non valide de netlist

10.2.3 fr.n7.idm.layout

Description : Métamodèle Layout pour le placement physique.

Fichiers importants :

- `model/layout.ecore` : métamodèle (importe `netlist.ecore`)
- `model/layout.genmodel` : génération

10.2.4 fr.n7.idm.bom.acceleo

Description : Transformation Acceleo pour générer la BOM à partir d'une netlist.

Fichiers importants :

- src/fr.n7.idm.bom.acceleo.main/GenerateBOM.java : classe principale
- generateBOM.mtl : template Acceleo de génération
- tasks/netlist_BOM.dot : fichier de sortie exemple

10.2.5 fr.n7.idm.design

Description : Éditeurs graphiques Sirius pour Netlist et Layout.

Fichiers importants :

- description/netlist.odesign : spécification Sirius pour netlist

10.2.6 fr.n7.idm.catalogatl.atl

Description : Transformations ATL pour le catalogue.

Fichiers importants :

- XtoCatalogue.atl : transformation ATL
- out.xml : résultat de transformation

10.2.7 fr.n7.idm.netlist.acceleo

Description : Transformations Acceleo pour la netlist.

Fichiers importants :

- tasks/netlist.dot : génération au format DOT

10.2.8 fr.n7.idm.catalog.acceleo

Description : Génération et validation pour le catalogue.

Fichiers importants :

- Catalogue.pdf : documentation générée
- CatalogueValide.dot : validation en DOT

10.2.9 fr.n7.idm.catalog.xtext

Description : Syntaxe textuelle Xtext pour le catalogue de composants.

Fichiers importants :

- `src/fr.n7.idm.catalog/Catalog.xtext` : grammaire Xtext définissant la syntaxe textuelle
- `GenerateCatalog.mwe2` : workflow de génération MWE2

10.3 Projets de tests

10.3.1 fr.n7.idm.tests

Description : Projet contenant les modèles exemples et tests.

Fichiers importants :

- `BOM.pdf` : BOM générée
- `circuit.pdf` : schéma du circuit
- `test.catalog` : catalogue exemple
- `test1.catalog, test5.netlist` : autres exemples
- `layoutfinal.layout` : layout exemple
- `NetlistFinal.netlist` : netlist finale

10.3.2 fr.n7.idm.test2

Description : Projet de tests supplémentaires.

Fichiers importants :

- `catalog3.catalog` : catalogue de test
- `NetlistValid.netlist` : netlist valide
- `test3.netlist` : netlist de test

11 Syntaxe textuelle (Xtext)

En complément des éditeurs graphiques, nous avons développé une syntaxe textuelle avec Xtext pour une saisie rapide.

11.1 Exemple de définition de catalogue

```
catalog MyCatalogue {  
  
    component Resistor {  
        id "RES001"  
        manufacturer "Vishay"  
        footprint RECTANGLE size 6.0 x 3.0  
        ports {  
            p1 (0.0, 1.0) type BIDIRECTIONNEL,  
            p2 (6.0, 1.0) type BIDIRECTIONNEL  
        }  
        metadata {  
            value = "1k0hms" as TEXTE  
        }  
        constraints {  
            emptyArea safetyZone : 1.5 mm  
        }  
    }  
  
    component Led {  
        id "LED001"  
        manufacturer "Kingbright"  
        footprint CERCLE size 5.0 x 5.0  
        ports {  
            anode (1.0, 2.0) type ENTREE,  
            cathode (3.0, 2.0) type SORTIE  
        }  
        constraints {  
            redundancy security_check : min 1 max 10  
            edge bottom_limit : 5.0 mm from BAS  
        }  
    }  
}
```

Listing 1: Exemple de syntaxe Xtext pour un catalogue de composants

12 Couverture fonctionnelle

Le tableau suivant récapitule la couverture des fonctionnalités attendues dans la spécification du projet.

Fonctionnalité	État	Remarque
F1 - Catalogues de composants		
F1.1 Métadonnées composants		Nom, fabricant, ID implémentés avec classe Metadonnee extensible (clé-valeur)
F1.2 Empreinte composant		Classe Empreinte avec largeur/hauteur et formes (RECTANGLE, CERCLE, OVALE)
F1.3 Ports sur empreinte		Classe Port avec positions (x,y) et types (ENTREE, SORTIE, BIDIRECTIONNEL)
F1.4 Contraintes composants		5 types implémentés : ContrainteAireVide, ContrainteDistance, ContrainteProximiteBord, ContrainteRedondance, ContrainteMetadonnee avec hiérarchie ET/OU/NON via ContrainteCombinee
F1.5 Interface catalogue		Double interface : éditeur Xtext textuel + éditeur arborescent EMF
F2 - Netlist		
F2.1 Visualisation netlist		Génération automatique via Acceleo : netlist.dot → circuit.pdf avec symboles électriques
F2.2 Interface graphique Sirius		Éditeur Sirius avec palette : création composants, connexions par nets, symboles standards
F2.3 Commentaires		Classe Commentaire sur instances et netlist avec attribut texte
F2.4 Export BOM		Template Acceleo netlistBOM.mtl générant BOM.pdf avec références, composants, fabricants, empreintes et métadonnées
F3 - Layout		
F3.1 Création layout		Métamodèle complet avec éditeur arborescent EMF
F3.2 Structure multicouche		Classe Board contenant Couche avec spécialisations CoucheExterne (max 2 par board) et CoucheInterne
F3.3 Placement composants		Classe ComposantPlace avec coordonnées (x,y) et rotation sur couches externes uniquement
F3.4 Pistes conductrices		Classe Piste avec liste de Points pour tracé sur toutes couches
F3.5 Vérification cohérence		LayoutValidator vérifie cohérence Layout Netlist et respect contraintes du catalogue (aires vides, distances, bords)
F3.6 Export images layout		Métamodèle complet permettant export, mais visualisation graphique Sirius non implémentée

Bilan de la couverture :

Fonctionnalités supplémentaires : Au-delà des exigences minimales, nous avons implémenté :

- Système complet de validation avec 31 contraintes Java réparties sur 3 validateurs
- Transformations ATL (M2M) pour interopérabilité
- Génération automatique de documentation (DOT, PDF)
- Syntaxe textuelle Xtext pour saisie rapide de catalogues

Limitation unique : L'éditeur graphique Sirius pour le Layout (F3.6) n'a pas été complété. Le méta-modèle Layout est fonctionnel et permet la modélisation complète du placement physique via l'éditeur arborescent, mais la visualisation graphique interactive des couches et composants placés reste à implémenter.

13 Conclusion

Ce projet d'Ingénierie Dirigée par les Modèles nous a permis de développer une solution complète pour la spécification et la validation de circuits électroniques, structurée autour de trois métamodèles complémentaires : **Catalog**, **Netlist** et **Layout**.

13.1 Réalisations

Nous avons mis en place une chaîne d'outils intégrée comprenant :

- Trois métamodèles Ecore interconnectés avec validation Java
- Un système complet de validateurs Java (CatalogValidator, NetlistValidator, LayoutValidator) implémentant plus de 30 contraintes métier
- Un éditeur graphique Sirius pour la création visuelle de netlists
- Une syntaxe textuelle Xtext pour la saisie rapide de catalogues
- Des transformations Aceleo (M2T) pour la génération automatique de BOM et documentation
- Des transformations ATL (M2M) pour l'interopérabilité entre formats

13.2 Apports de l'approche MDE

L'Ingénierie Dirigée par les Modèles a démontré son efficacité dans ce contexte :

Séparation des préoccupations : Les trois métamodèles permettent de traiter indépendamment les aspects logiques (Netlist) et physiques (Layout) tout en partageant un catalogue commun de composants réutilisables.

Validation précoce : Les validateurs Java détectent les erreurs de conception avant la fabrication physique, réduisant considérablement les coûts de correction. Les contraintes couvrent l'unicité des identifiants, la cohérence des connexions, le respect des règles de placement, et la validation des contraintes géométriques.

Automatisation : La génération automatique de documentation (BOM, schémas DOT/PDF) élimine les tâches répétitives et garantit la cohérence entre modèle et documentation.

Réutilisabilité : Les composants définis dans le catalogue peuvent être instanciés dans de multiples circuits, augmentant la productivité.

Feedback immédiat : L'intégration des validateurs dans Eclipse permet une détection en temps réel des erreurs avec affichage dans la vue Problems, guidant le concepteur vers des modèles valides.

13.3 Limitations et perspectives

Notre travail présente néanmoins certaines limitations. La principale concerne la **visualisation graphique du Layout** : malgré la complétude du métamodèle Layout et sa capacité à modéliser les couches, les composants placés et les pistes, nous n'avons pas pu implémenter l'éditeur graphique Sirius correspondant dans les délais impartis. Cette visualisation aurait permis une conception interactive du placement physique et une vérification visuelle des contraintes géométriques (chevauchements, distances minimales, aires de sécurité).

D'autres améliorations futures pourraient inclure :

- Export vers des formats industriels (Gerber, KiCad)
- Optimisation automatique du routage
- Intégration de simulations électriques (SPICE)
- Visualisation 3D du PCB final
- Extension du système de validation avec des suggestions automatiques de corrections

13.4 Bilan

Ce projet illustre concrètement l'importance de la modélisation dans l'ingénierie des systèmes complexes. En structurant les connaissances métier dans des métamodèles précis, en implémentant un système robuste de validation par validateurs Java, et en automatisant les processus de génération, l'approche MDE améliore significativement la qualité, la fiabilité et la maintenabilité des conceptions électroniques.

Au-delà des aspects techniques, ce travail nous a permis de maîtriser les technologies clés de l'écosystème Eclipse/EMF (Ecore, Sirius, Xtext, Acceleo, ATL) et de comprendre les enjeux réels de l'ingénierie dirigée par les modèles dans un contexte industriel. L'implémentation de validateurs Java nous a notamment permis d'appréhender la différence entre contraintes structurelles (définies dans Ecore) et contraintes sémantiques (vérifiées par code).

L'approche MDE n'est pas un simple exercice académique : elle répond à des problématiques concrètes où la complexité croissante des systèmes électroniques (IoT, systèmes embarqués critiques) nécessite des méthodes formelles pour garantir fiabilité et time-to-market.